

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 01 · LECTURE NOTES

The Shape of the Problem

What determines whether it can be solved.

MODULE OUTCOMES

- State the general optimisation problem and its components.
- Explain why convexity is the dividing line.
- Classify the standard problem families by difficulty.
- Explain what local information can and cannot establish.
- Recognise optimisation problems in disguise.

1 The general problem

$$\min f(\mathbf{x}) \quad \text{s.t.} \quad g_i(\mathbf{x}) \leq 0, \quad h_j(\mathbf{x}) = 0$$

Three components: an **objective** f to minimise (maximisation is minimising $-f$, so nothing is lost), **inequality constraints**, and **equality constraints**. The constraints together define the **feasible set**. **The difficulty of the problem depends on the shape of f and of the feasible set, not on the number of variables** — a convex problem in a million variables is routinely solved, and a carefully constructed integer problem in fifty variables can be open. **That inversion of the expected relationship between size and difficulty is the first thing to internalise**, because it redirects effort from reducing dimensionality to improving structure.

THE DIVIDING LINE IS CONVEXITY, NOT LINEARITY

For a convex problem, every local minimum is a global minimum. There are no other candidates to rule out — the geometry forbids them.

And that is the only reason local methods work. Gradient descent sees a neighbourhood and nothing more; **convexity is precisely what licenses the inference from 'I cannot improve locally' to 'this is the best point anywhere'**. Without it the inference is invalid, and every algorithm in this course that offers a guarantee offers it on the strength of this single property.

Without convexity, finding a global minimum is intractable in general. You cannot rule out a better point in a region you have not examined, and the number of regions is exponential.

So the first question about any optimisation problem is 'is it convex?', and the answer determines what kind of guarantee is available at all — before any question of which algorithm to use. **Linearity is a special case of convexity and is not the relevant boundary**, which is a correction worth making early because the historical development (linear programming first) suggests otherwise.

2 The hierarchy

Class	Form	Difficulty
Least squares	minimise $\ Ax - b\ ^2$	Closed form — the normal equations, though not computed that way (CSCE 633 Module 02 §1). A solved problem.
Linear program	Linear objective, linear constraints.	Polynomial time , and modern solvers handle millions of variables. Modules 03, 04, 07.
Quadratic program	Convex quadratic objective, linear constraints.	Polynomial . Support vector machines are exactly this (CSCE 633 Module 07), as are many control and finance problems.
Conic and semidefinite programs	Optimisation over convex cones; constraints requiring a matrix to be positive semidefinite.	Polynomial but slower . The source of the strongest known relaxations for hard combinatorial problems (Module 10).
General convex	Convex f over a convex set.	Polynomial , given an oracle for f and its subgradients — the ellipsoid method establishes this, and interior point methods make it practical.
Integer program	Some or all variables constrained to integers.	NP-hard — and solved routinely in practice (Module 09). See the callout.
General non-convex continuous	No structure assumed.	No global guarantee available . CSCE 636's territory , where the practice runs well ahead of the theory.

NP-HARD DOES NOT MEAN UNSOLVABLE

NP-hardness is a statement about worst-case behaviour across all instances, including adversarially constructed ones that arise nowhere outside complexity proofs (CSCE 629).

Real instances have structure. Sparsity, symmetry, block structure, near-integral relaxations, and good warm starts — and commercial solvers exploit all of it aggressively, with decades of accumulated heuristics.

So integer programs with hundreds of thousands of variables are solved to optimality daily in logistics, scheduling, and manufacturing, while carefully constructed instances with a few dozen variables remain intractable.

The practical question is therefore not 'is this problem class NP-hard?' but 'does the solver terminate on my instances?' — which is an empirical question answered by trying it, not a theoretical one answered by classification. **CSCE 629's complexity classes bound the worst case, not your Tuesday**, and treating NP-hardness as a reason not to attempt a formulation is a common and costly error.

3 What local information establishes

A GRADIENT IS A LOCAL STATEMENT

The gradient gives the direction of steepest increase of f at a point — *at that point*. It is a derivative: it describes the function's behaviour in an infinitesimal neighbourhood and says nothing whatsoever about the function elsewhere.

So a point where the gradient vanishes is *stationary*, which may be a local minimum, a local maximum, or a saddle point — and in high dimensions, overwhelmingly the last (CSCE 636 Module 03 §4).

The Hessian distinguishes them locally: positive definite means a strict local minimum, negative definite a maximum, indefinite a saddle. **This is still entirely local information** and establishes nothing about the global landscape.

Convexity is the bridge between the two. For a convex function, stationary implies globally minimal — and that single implication is what converts a local algorithm into a global method with a certificate. **Everything in Modules 05 through 07 depends on it**, and Module 02 is about establishing it.

4 Optimisation problems in disguise

- **Fitting a model** — minimise a loss over parameters (CSCE 633 Module 01). The entire previous two courses are instances of this course's subject.
- **Scheduling, routing, assignment, packing, and covering** — integer programs almost without exception, and the largest commercial application of the subject.
- **Maximum likelihood estimation** — minimise the negative log-likelihood (CSCE 633 Module 10 §1), which is convex for the exponential family and not in general.
- **Shortest path, maximum flow, and bipartite matching** — **linear programs whose optima happen to be integral** (CSCE 629, and Module 11), which is why they are easy when general integer programs are not.
- **Inverse rendering** — minimise the difference between a rendered image and a photograph, over scene parameters (CSCE 636 Module 11). Non-convex, and solved by gradient descent anyway.
- **And portfolio selection, optimal control, and economic equilibrium** — which is why this subject is shared between computer science, operations research, engineering, and economics, and why its vocabulary comes from all four. **Recognising a problem as an optimisation problem is most of solving it**, because at that moment a century of machinery becomes applicable.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boyd & Vandenberghe — Convex Optimization, chapter 1 (free)	The §1 and §2 framing , and the clearest available statement of why convexity rather than linearity is the boundary.
Stanford EE364A — lecture 1 (free video)	Boyd lecturing the same material. Worth watching alongside the chapter.
MIT 15.093 — introduction and problem classification (free)	The §2 hierarchy with more emphasis on the linear and integer end.
Bixby — Solving Real-World Linear Programs: A Decade and More of Progress (free)	Evidence for the §2 callout — what solvers actually achieve on real instances, and how far that is from the worst case.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write three problems you care about in the §1 standard form.
- 2 **For each, classify it** into one of §2's seven families.
- 3 Solve a least squares problem, a linear program, and a small integer program with a solver, and compare the solve times.
- 4 **Construct a small integer program the solver struggles with**, and a large one it solves instantly. Explain the difference.
- 5 **Plot a non-convex function in one dimension** and run gradient descent from many starting points. Record the distinct minima found.
- 6 Do the same for a convex function and confirm every start converges to the same point.
- 7 **Find a stationary point that is a saddle** and verify with the Hessian.
- 8 Formulate shortest path as a linear program and verify the solution is integral.
- 9 **Express a problem from CSCE 633 in this course's notation** and say whether it is convex.
- 10 Install CVXPY and a solver, and reproduce one textbook example.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the general optimisation problem and say what determines its difficulty.
- 2 Why is convexity the dividing line, and why not linearity?
- 3 Name seven problem classes and their difficulty.

- 4 Why does NP-hardness not mean unsolvable in practice?
- 5 What does a gradient establish, and what does a Hessian add?
- 6 What does convexity let you conclude that you otherwise could not?
- 7 Name six problems that are optimisation problems in disguise.